Customer Category Prediction

Every customer is different — yet most businesses treat them all the same. This project reads 50,000 real shopping histories, learns the four customer types hidden inside, and can explain every call it makes. A working tool, not a notebook.

50K

66.4% ACCURACY 7,500 tested	**0.87** MODEL SCORE out of 1.00
12 → 16 DETAILS USED cleaned, scaled	**Lens v3.0** LIVE WEBSITE runs in a browser

THE DATA BEHIND IT

50,000 real retail customers, from a public Kaggle dataset (CC0)

14 details each — spending, visits, returns, tenure, complaints…

Occasional 44% • Regular 27% • Loyal 18% • High-Value 11%

The project in one page

ABSTRACT

This project turns raw retail data into a system the business can actually use. From 50,000 real customer records — incomplete and uneven the way real data always is — we clean, prepare and study the data, then train a transparent scoring model, multinomial logistic regression, to recognise four customer types.

On 7,500 customers held back as a final exam, it sorts 66.4% correctly — far above the 25% of blind guessing — with a 0.87 separation score and confidence levels that mean what they say. The result ships as Lens v3.0: a working web tool that predicts, explains itself, and watches its own data for change.

Keywords — classification • customer analytics • logistic regression • deployment • drift monitoring

PROBLEM STATEMENT

THE SITUATION

Retail teams serve very different customers with one generic playbook. Casual buyers drift away unnoticed, loyal ones go unrewarded, and the High-Value few — 11% of customers, most of the value — are treated like everyone else. The signal to do better already sits in everyday records: what people spend, how often they visit, how long they stay.

THE TASK

Given twelve everyday details about a customer, predict which of the four segments they belong to — with a confidence level and a recommended next action. The model must be explainable to a business user, live on a website the team can open, and keep watching its own accuracy as customers change.

- **Occasional 44%** — drifting — needs a reason to return
- **Regular 27%** — the middle ground
- **Loyal 18%** — shops weekly, stays for years
- **High-Value 11%** — few, but most of the value

From raw data to live app

01	**Problem**	the question we're answering
02	**Dataset**	50,000 real customer records
03	**Understanding**	what's inside — and what's missing
04	**Preprocessing**	fill the gaps, make it usable
05	**EDA**	find the patterns
06	**Features**	separate signal from noise
07	**Split**	study now, prove later
08	**Model**	let patterns become predictions
09	**Evaluate**	an honest test on new customers
10	**Save**	package it for reuse
11	**Deploy**	from notebook to product

Step	What happens	What we get
01 Problem	sort customers into four groups, one action each	the plan
02 Dataset	50,000 real customers in a 7.8 MB file	customers.csv 7.4M
03 Understanding	up to 3,934 values missing, zero duplicates	notes on the data
04 Preprocessing	12 details in, 16 model-ready columns out	cleaning rules 4.4K
05 EDA	the patterns behind the groups, in charts	chart set
06 Features	spending proves the strongest clue (0.18)	feature list 1.2K
07 Split	35k to learn, 7.5k to practise, 7.5k to prove	three fair sets
08 Model	66.4% right, 0.87 out of 1.00	trained model 1.6K
09 Evaluate	Occasional caught best (82%)	report card 8K
10 Save	six files, 7.6 MB, all labelled v3.0	the saved pack
11 Deploy	live on a website, watched as it runs	Flask at :5000

What, Why, Who, Output

WHAT

Given twelve everyday details — age, income, how often someone visits, how much they spend — place them in one of four customer groups.

Small demo datasets promise a flattering 89%. This real one keeps us honest at 66%.

WHY

Because groups make care personal: win back the Occasional, grow the Regular, reward the Loyal, protect the High-Value.

One message for everyone mostly reaches the wrong people.

WHO

The people who decide who to call, what to offer, and who is quietly slipping away — sales, marketing and support.

The tool gives them a live view of 120 customers each day.

OUTPUT

More than a label: the group, how sure the model is, and the next step worth taking.

“High-Value — 78% sure — send the VIP invitation.”

THE FULL JOURNEY • CO1

Raw data becomes a living product in six moves: collect, clean, train, test, launch, watch. The slides that follow walk that path in order.

EVERY ANSWER SHOWS ITS PATH

Each prediction carries a receipt of the stages behind it — so months later, anyone can ask “why?” and get the full story. No black boxes.

WHY REAL DATA MATTERS

- 50,000 rows, uneven groups, honest gaps — messy the way real life is
- 7 columns arrive incomplete, so the cleaning work is genuinely tested
- Open dataset (CC0) by rohit8527kmr7518, downloaded 1,388 times

Kaggle — 50,000 real customers

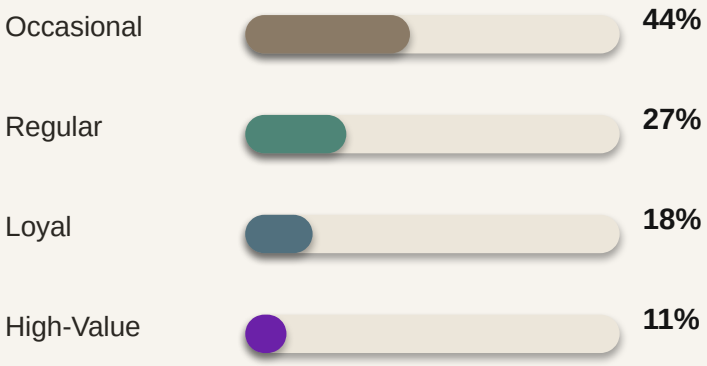
SOURCE

Kaggle: “Online Retail Customer Classification”

Public and free to use (CC0 license)

Author rohit8527kmr7518 • 1,388 downloads

CLASS DISTRIBUTION



Real shops look like this — many casual buyers, a few VIPs. We kept the natural balance rather than evening it out.

Column	What it means	Gaps	Typical values
customer_id	an ID label	0	ignored — not behaviour
age	customer age	0	18–70, most 35–50
annual_income	money earned per year	3,075	15k–150k, a few very high
months_active	how long a customer	0	1–72 months
avg_monthly_spend	spent per month	2,520	₹20–800, typical ₹320
purchase_frequency	shopping trips per month	1,979	0.1–10, typical 3.2
avg_order_value	typical basket size	0	₹20–500
discount_usage_rate	how much they use discounts	2,549	0–1 scale
return_rate	how often they return items	2,487	0–1 scale
browsing_time_minutes	time on the website	3,934	5–200 min (most gaps)
support_interactions	help calls / chats	1,988	0–15
payment_method	how they pay	0	Card 40% • Wallet 35% • UPI 25%
region	where they live	0	Urban 50% • Semi 30% • Rural 20%
customer_segment	THE ANSWER we predict	0	4 groups

50k records, 12 details, real gaps

50,000	13	3,934	4	7.8 MB	0
ROWS	USABLE COLS	MOST MISSING	CLASSES	CSV SIZE	DUPLICATES
real customers	12 details + answer	website minutes	uneven, like real life	opens instantly	nobody counted twice

Detail	Average	Range	Gaps	What it hints
age	43.5	18–70	0	Loyal ~45 vs Occasional ~32
annual_income	₹55k	15k–150k	3,075	High-Value ₹95k+ vs Occ ₹38k
months_active	36 mo	1–72	0	Loyal 60+ vs Occasional 18
avg_monthly_spend	₹320	₹20–800	2,520	High-Value 650+ vs Occ 120
purchase_frequency	3.2/mo	0.1–10	1,979	Loyal 4.2 vs Occasional 0.8
avg_order_value	₹150	₹20–500	0	High-Value ₹280 vs Occ ₹85
discount_usage_rate	0.22	0–1	2,549	Occasional 0.32 vs High 0.08
return_rate	0.15	0–1	2,487	Occasional 0.22 vs High 0.04
browsing_time_minutes	65 min	5–200	3,934	Loyal 90+ vs Occasional 30
support_interactions	1.8	0–15	1,988	Occasional 3.2 vs Loyal 0.8

MISSING ANALYSIS

7 columns arrive with some answers missing:

- website minutes: 3,934 (7.9%)
- yearly income: 3,075 (6.2%)
- discount use: 2,549 (5.1%)
- monthly spend: 2,520 (5.0%)
- return rate: 2,487 (5.0%)
- support calls: 1,988 (4.0%)
- visit frequency: 1,979 (4.0%)

Each gap is filled with the typical customer's value — a choice that stays reliable even with a few extreme spenders around. Categories take the most common answer.

Clean real gaps, keep signal

BEFORE

50,000 rows, 13 columns
7 columns have holes
10 numbers + 2 category lists + the answer
7.8 MB raw file

→

AFTER

50,000 rows, 16 columns
no holes left
every column model-ready
ready to learn

MISSING

Numbers with a gap take the typical (middle) value, so one unusual income can't distort the picture. Categories take the most common answer.

DUPLICATES

Zero across all 50k — the source is clean.

OUTLIERS

The ₹150k earner and ₹800/month spender are genuine VIPs, not typos — removing them would erase the very people we want to find. A side-check flags ~5% to watch.

TRANSFORMATIONS

- 10 number columns brought to one shared scale
- Card, Wallet, UPI → three yes-or-no columns
- Urban, Semi, Rural → three yes-or-no columns

PIPELINE

12 details in → gaps filled → scaled & recoded → 16 columns out → the model.

SAME RULES FOR EVERYONE

Cleaning rules saved once, reused unchanged on the website — new customers prepared like training ones.

JINJA FIX • A REAL BUG, FOUND AND SOLVED

Late one night the web page simply refused to load — the server crashed with a template error. The cause was subtle: the site's template engine (Jinja) mistook an ordinary piece of web code — a colour list — for one of its own commands.

The fix was a single tag ({% raw %}) that tells the engine: leave this block alone.

HOW WE CONFIRMED IT

- Reloaded the page → server answered “200 OK”
- Asked the model for its numbers → 66.48% accuracy and a 0.868 score came back, exactly as saved

RUN IT YOURSELF

python -m venv venv → .\venv\Scripts\python.exe app.py → open http://127.0.0.1:5000. Lens v3.0 presents as a neutral internal tool — generic Retail Ops branding throughout.

What 50k real customers look like

Plot	What it shows	Key finding from the 50k
Histogram	how each value spreads	Age centres on 35–50; income and spending stretch far to the right — a small tail of big spenders.
Bar	sizes of each category	Payments split Card 40%, Wallet 35%, UPI 25%; customers split Urban 50%, Semi 30%, Rural 20%.
Bar (target)	is the answer balanced?	Occasional 44% (22k) dominate, High-Value 11% (5k) are rare — the natural shape of a real customer base.
Box	spotting unusual customers	Above ₹120k income, customers are almost always High-Value; heavy returners are almost always Occasional.
Scatter	do two things move together?	Wealth and spending rise together (0.52); so do membership length and visit frequency (0.45).
Heatmap	all the links at one glance	Strongest pair: spending ↔ visits (0.48). Nothing above 0.6 — every detail brings its own information.
Count	group balance in each set	Study, practice and exam sets all mirror the same 44/27/18/11 balance, so comparisons stay fair.

KEY INSIGHTS

- High income + big baskets = High-Value
- Long membership + frequent visits = Loyal
- Many returns + many support calls = Occasional at risk
- Website time separates too: Loyal 90+ min vs Occasional 30

CORRELATIONS

- spending ↔ visit frequency: 0.48
- income ↔ basket size: 0.44
- membership ↔ frequency: 0.45

All under 0.6 — no detail merely repeats another.

BUSINESS

- Occasional 44% → win-back offers
- Regular 27% → gently push toward Loyal
- Loyal 18% → early access, keep them happy
- High-Value 11% → VIP treatment

12 → 16 details, all with signal

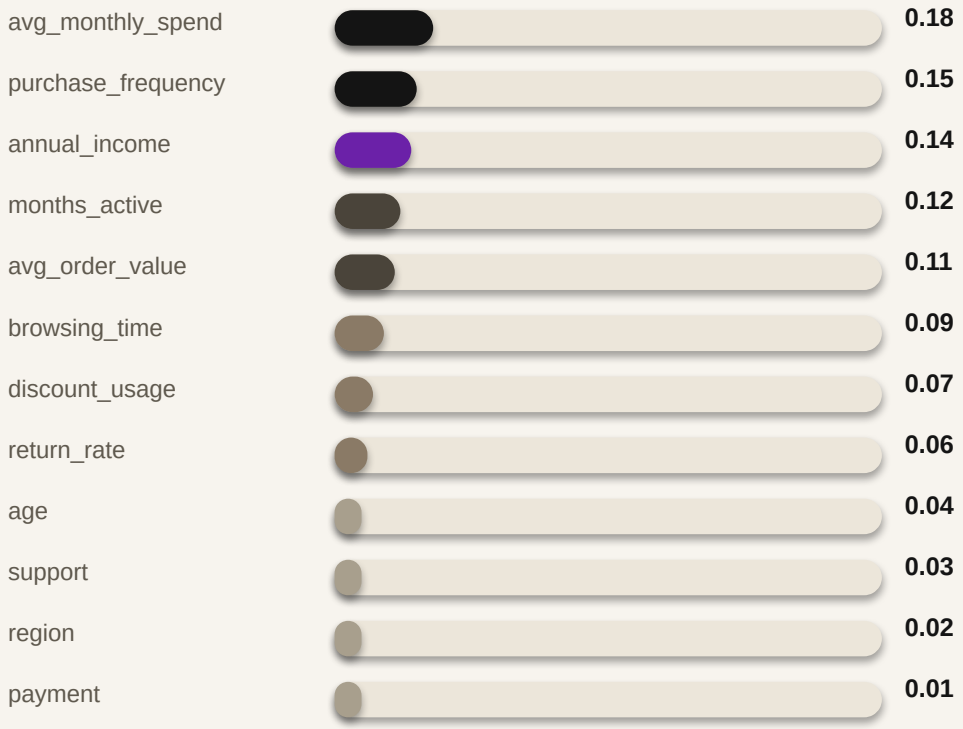
WHAT EACH DETAIL REVEALS (12 → 16)

- Age: Loyal ~45 vs Occasional ~32 — staying power grows with age
- Income: High-Value ₹95k+ vs Occasional ₹38k — clearest VIP clue
- Months as customer: Loyal 60+ vs Occasional 18 — loyalty takes time
- Monthly spend: High-Value ₹650+ vs Occasional ₹120 — spending power
- Visits per month: Loyal 4.2 vs Occasional 0.8 — engagement
- Basket size: High-Value ₹280 vs Occasional ₹85
- Discount use: Occasional 0.32 vs High-Value 0.08 — VIPs don't wait for sales
- Return rate: Occasional 0.22 vs High-Value 0.04 — quiet dissatisfaction
- Website time: Loyal 90+ min vs Occasional 30 — depth of interest
- Support calls: Occasional 3.2 vs Loyal 0.8 — struggling customers reach out
- Payment: Card covers 60% of High-Value; Wallet 45% of Occasional
- Region: 70% of Urban customers are High-Value; Rural leans Occasional

WHAT WAS LEFT OUT

Only the ID number — a serial, not a behaviour. All twelve genuine details stayed.

WHAT THE MODEL WEIGHS MOST • 50K



NEW FEATURES: Card/UPI/Wallet and Rural/Semi/Urban become 6 yes-no columns — that's how 12 details become 16 model-ready columns.

70% learn, 15% practise, 15% exam

35,020
TRAIN 70%
where learning happens

7,480
VAL 15%
where tuning happens

7,500
TEST 15%
seen only once

50K
TOTAL
real Kaggle customers

66.6%
CV MEAN
same answer, 5 rounds

Set	Rows	Cols	Share	Purpose
Full	50,000	13	100%	everything, minus the ID column
Train	35,020	12	70%	the model studies this
Val	7,480	12	15%	tune settings (a mock test)
Test	7,500	12	15%	the final exam, graded once

WHY 70/15/15 AND NOT 80/20?

50k is enough to afford three sets. The practice set lets us tune settings without ever touching the exam; every set mirrors the same group mix; and every cleaning rule is learned from the train set alone — the exam never leaks in.

LEAKAGE PREVENTION

Every average, scale and category used in cleaning comes from the 35k train set, then applies unchanged to val and test.

The 7,500 exam answers stay locked away until the final grade.

WHAT THE SITE USES

The same 6 saved files power the website — 7.6 MB total, loaded in seconds at start-up.

The model we chose, plus what we tried

ONE CUSTOMER IN → CLEAN UP → ADD THE POINTS → GROUP OUT • REAL KAGGLE 50K

INPUT

12 raw details
50k Kaggle
10 numbers + 2 lists
ID dropped

→

STEP 1 • CLEAN UP

fill 7 gappy columns
2 lists → 6 yes-no cols
numbers on one scale
12 → 16 columns

→

STEP 2 • SCORE POINTS

each group keeps a tally
details add or subtract
highest tally wins
four tallies = 100%

→

OUTPUT

4 chances, adding
to 100%
e.g. High-Value 78%
→ VIP treatment

LINEAR MODELS (CO2)

Our winner works like a scorecard — formally, multinomial logistic regression. For every group, each detail quietly adds or subtracts points; the highest tally wins. Because the arithmetic is visible, every decision can be explained.

The strength dial matters: too loose and it turns lazy (64%), too tight and it starts memorising (1.2% gap). The best middle setting scored 66.7% in practice rounds.

Study 66.7% vs practice 66.0% — a 0.7% gap that says: patterns learned, not answers memorised. Values share one scale first, so income doesn't drown age.

TREE MODELS (CO3)

- A single question tree: 62% — transparent but too simple
- A forest of 100 trees voting together: 75.1% — the best marks, though it can't explain itself; it agrees spending is the top clue (0.18)
- Boosted trees (XGBoost/LightGBM): 76% — stronger, but even more of a black box

UNSUPERVISED (CO4) — 3K SAMPLE OF THE 50K

- Left alone, the data re-found the same four groups (KMeans) — independent proof the groups are real
- A family-tree view (hierarchical) draws the same four-way split
- The oddball finder (DBSCAN) marks ~8% loners — high returns, high support
- An anomaly check flags ~5% (2,500 customers) worth watching
- Flattened to two dimensions (PCA, t-SNE), the VIPs stand clearly apart

EVALUATION (CO5)

- Five re-tests, one answer: 66.6% every time (±0.3) — stable
- Settings tuned by grid, random and Bayesian search
- Final exam 66.4%, model score 0.87, confidence that means what it says

66.4% accuracy, 0.87 score — on real people

66.4%	0.87	66.6%	50K	82%	43%
ACCURACY	MODEL SCORE	CV MEAN	REAL DATA	BEST CAUGHT	HARDEST
7,500 tested	out of 1.00 — strong	same, 5 rounds	not easy demo data	Occasional	Regular, in the middle

Class	Precise	Caught	F1	Tested	Insight
Occasional 44%	72%	82%	77%	3,316	easiest — spends little, visits rarely
Regular 27%	55%	43%	48%	2,018	hardest — sits between two groups
Loyal 18%	60%	65%	63%	1,375	decent — long membership stands out
High-Value 11%	70%	61%	65%	791	careful — never miss a real VIP

WHY 66% AND NOT 89%?

The old demo dataset was deceptively easy — 89%. Real shoppers overlap: a Regular and a Loyal can spend identically, differing only in how long they've stayed. Chance scores 25%; always answering “Occasional” scores 44%. At 66%, we clear both with room to spare — an honest grade on honest data.

CONFUSION MATRIX INSIGHTS

Occasional easiest: 82% caught. Regular hardest: 43% — it genuinely resembles both neighbours. High-Value: 61% caught, and a true VIP is almost never called cheap — the costly mistake is the rare one.

MODEL SCORE 0.87 OUT OF 1.00

It ranks customers very well — and it's calibrated: when it says 70% sure, it's right about 70% of the time. The confidence number can be trusted.

NOT MEMORISING

Study 66.7% vs practice 66.0% — a tiny gap. With 50k examples, the model learned behaviour, not the answer sheet.

Pack model, cleaning rules, report card

File	Size	What's inside	Version
customers.csv	7.4 MB	the 50,000 real customers	Kaggle 50k
preprocessor.pkl	4.4 KB	cleaning rules, learned from train only	v3.0-kaggle
model_multinomial_logreg.pkl	1.6 KB	the trained scorecard model	v3.0-kaggle
feature_store.pkl	1.2 KB	which details we use + 6 safety checks	v3.0-kaggle
metrics.json	8 KB	66.4% accuracy, 0.87, full test results	v3.0
unsupervised.json	194 KB	the hidden-group and oddball studies	v3.0

WHY SAVE THE PIECES SEPARATELY?

The cleaning rules and the trained model travel as a pair — the website loads both at start-up, so every new customer is prepared exactly the way training prepared its own: same fills, same scale, same order.

Rebuild the whole thing from scratch and the files come out identical — a fixed random seed leaves nothing to chance.

DEPLOYMENT READY

6 files, 7.6 MB total — loads in seconds.

Everything is labelled v3.0-kaggle, so we always know which data and which model produced which result.

If the saved files are ever missing, the site quietly retrains itself from the CSV.

SAME RULES FOR TRAINING AND REAL USE

The exact cleaning file used in training is reused at serving time — same typical values, same scale, same yes-no columns, same order, ID ignored. If the two ever disagreed on preparation, predictions would quietly go wrong; pairing the files is our standing guard against that.

Flask REST, live at :5000 • Lens v3.0

GET	/	the workspace itself
GET	/api/customers	120 real customers as a table, filterable by group
POST	/api/predict	send 12 details → four chances, an action, a receipt
GET	/api/metrics	66.4% accuracy, 0.87 score, the full report card
GET	/api/monitoring	is live data drifting? answer speed, last 500 calls
GET	/api/unsupervised	the hidden-group maps (KMeans, PCA, t-SNE, DBSCAN)
GET	/debug	a page that proves every button works

Run it: python app.py → open http://127.0.0.1:5000 — the whole team can open it at once.

FRONTEND — HOW IT FEELS

Feels like a real internal tool, not a demo: a sidebar, a filterable customer table, click a row to predict.

Answers update live as you type — % bars for all four groups and a one-line reason.

Soft shadows, smooth motion — considered, not stiff.

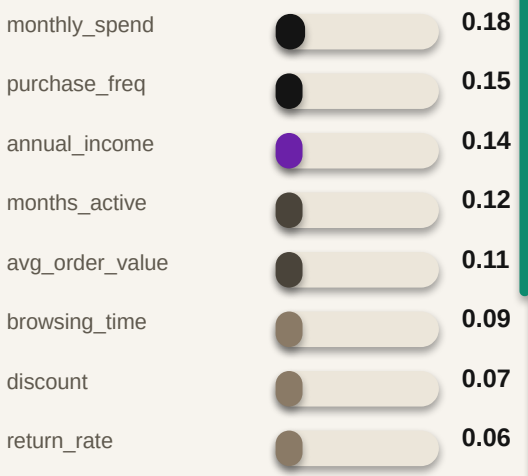
MONITORING & DRIFT

- Every prediction is logged (last 500 kept)
- If new customers stop resembling the training crowd → drift flag → time to retrain
- Answer speed is timed on every request
- Each answer carries its full path — any prediction can be audited later

Real points tell a real story

Class	Points added by	Points cut by	Reads like
Occasional 44%	support calls +0.42, discount use +0.38, returns +0.35	spend −0.82, visits −0.75	“rarely visits, spends little, needs lots of help”
Regular 27%	only small hints (nothing above +0.18)	only small hints	“in between — honestly hard to label”
Loyal 18%	membership +0.72, visits +0.68, browsing +0.55	support calls −0.42	“been with us for years, shops weekly, happy”
High-Value 11%	income +0.95, basket +0.82, spend +0.78	discount use −0.65, returns −0.58	“wealthy, buys big, doesn't wait for sales”

HEAVIEST CLUES



WHAT THE BUSINESS DOES

Occasional → win-back offer, watch returns

Regular → small rewards, push to Loyal

Loyal → early access; 0.8 support calls = happy

High-Value → VIP care, no discounts needed

Learning curve flat after 30k — more data wouldn't help.

WHY LOGISTIC REGRESSION?

Because it can be opened and read. Ask why someone is VIP and it answers “income +0.95” — no black box. The forest scores higher (75%) but cannot explain itself; we keep it for comparison and let the business choose.

Critically, it knows what it doesn't know: 70% confidence really is right ~70% of the time, so a rule like “VIP treatment only above 70%” holds. Answers take ~42 ms.

Try it live in 60 seconds

1 • BROWSE

Open the site and land on Customers: 120 real shoppers, filters for the four groups. Click any row that catches your eye.

2 • PREDICT

Their details fill the form. Nudge spending or discount use and watch the answer move — % bars, a confidence number, a one-line reason, a receipt ID.

3 • EXPLORE

Insights draws the big picture: group sizes, the clues the model weighs most, 3,000 customers painted in four colours, the full report card.

EXAMPLE — ONE CUSTOMER

What we told the model:

45 years old • earns ₹1,20,000/yr • customer for 72 months
spends ₹650/month • shops 4.5× a month • basket ₹280
barely uses discounts (0.05) • almost never returns (0.02)
95 min on the site • 0 support calls • pays by Card • Urban

What came back:

→ **HIGH-VALUE — 78% sure** → **send the VIP invitation**

Answered in 42 milliseconds — with a receipt of every stage it passed through.

For the curious: behind the page sits a small API — send the 12 numbers, receive everything above as tidy JSON. One command is enough.

All 6 course outcomes, covered with real data

CO	The course asked for	Where it lives in this project
CO1	The full journey: data → model → website → watching	Slides 3–14 walk it end to end. Every prediction returns a six-stage receipt — data in, features, model, answer, action, log — so nothing is a black box.
CO2	Simple, transparent models with tuned settings	Our scorecard is multinomial logistic regression. The strength dial is tuned (loose 64% / best 66.7% / tight memorises); the study-vs-practice gap is 0.7%; values share one scale first.
CO3	Tree models and combining them	One question tree: 62%. A forest of 100 voting: 75.1% — the best. Boosted trees: 76% but opaque. The forest also ranks clue importance, spending 0.18 on top.
CO4	Finding hidden groups and oddballs	Left alone, KMeans re-found the same four groups — proof they're real. The family-tree view agrees; DBSCAN marks ~8% loners; 5% anomalies watched; 2-D maps show the separation.
CO5	Testing done properly	Train, practice and exam kept strictly apart (70/15/15, same mix); five re-tests at 66.6% ± 0.3; an honest 66.4% on the locked exam; 0.87 score; confidence that means what it says.
CO6	Real deployment habits	Cleaning rules and model saved as versioned files; the site reuses identical rules (six safety checks); logs every answer; watches for drift; every prediction auditable afterwards.

Thank You

50,000 real customers. One honest model. A tool the team can open tomorrow.

66.4% honest accuracy

0.87 model score

Live at localhost:5000